

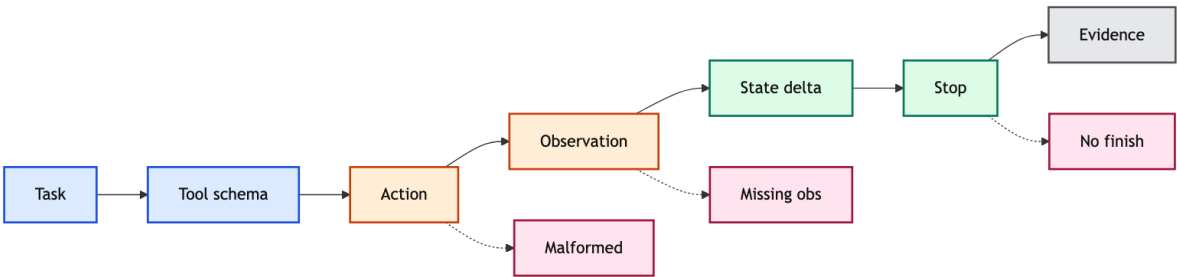
第 19 讲：轨迹完整性：工具格式、因果链与终止信号

Coding Agent Notes (June 2026)

系统位置	本讲位于 harness、context、long-running session 和 tool design 之间，专门解释 trajectory 作为训练、评测、回放和诊断对象的完整性要求。
已知前提	已经理解 action/observation/runtime/validation 的基本 agent loop，也知道 durable session 需要 event log 和 checkpoint。
新增问题	一个多轮工具调用轨迹什么时候仍然是可学习、可验证、可回放的对象；什么时候只是一段看似完整但会污染系统的噪声。
读完可回答	为什么 malformed tool call、缺失 observation、错误参数、坏终止信号和随机切碎的上下文都会破坏 coding agent。

1 核心图景

Trajectory 很容易被误解成“agent 跑完以后留下来的日志”。这个理解不够准确。对 coding agent 来说，trajectory 是一个协议对象：它把任务、工具 schema、模型动作、环境 observation、状态变化、终止信号和最后证据串成一条可检查的因果链。日志只要求记录发生过什么；trajectory 还要求这些记录能被训练器学习、被评测器判分、被 replay 系统复现、被诊断系统定位失败。



这张图里的每个节点都是契约的一部分。任务告诉模型要解决什么；tool schema 规定动作空间；action 是模型选择的外部行为；observation 是环境对行为的反馈；state delta 表示仓库、文件、shell、浏览器或 API 状态被怎样改变；stop 决定这条轨迹何时被判定；evidence 让外部系统知道它为什么通过或失败。任何一段断裂，都会改变这条数据的语义。

中心命题：

Trajectory integrity 不是日志美观问题，而是监督信号、评测信号和恢复信号是否仍然可信的问题。
一个 malformed call 如果被当作正常 action 学进去，模型学到的不是“修复代码”，而是“协议可以被破坏”。

本讲不重讲 #05 的 harness loop，也不重讲 #17 的持久化会话。这里关心的是更基础的对象质量：一条轨迹从数据、训练、运行到评测的各个环节是否仍然保持同一个含义。

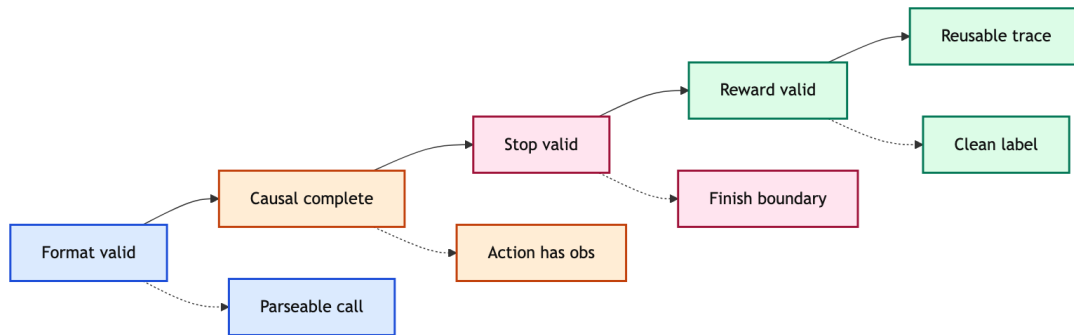
2 轨迹为什么不是普通日志

普通日志的主要任务是可观察：它告诉人或程序系统曾经发生了哪些事件。Trajectory 的要求更高，因为同一段记录会被不同模块重新解释。训练器把它看成监督样本；RL 系统把它看成 rollout；

benchmark 把它看成被判分的行为；debugger 把它看成失败因果链；long-running harness 把它看成恢复和 replay 的依据。

视角	需要 trajectory 提供什么	破坏后果
训练样本	schema、action、observation、answer 的一致关系	学到坏格式或错误因果
RL rollout	清楚的 reward boundary 和中间错误信号	奖励错配、超长 rollout、无效探索
评测对象	可判定的工具名、参数、顺序和最终结果	final score 难以解释
恢复配方 失败诊断	事件顺序、状态变化、未完成动作 每一步的证据和约束违反点	replay 不可复现或产生副作用 只能看到终局失败，找不到 root cause

因此，trajectory integrity 至少有四层：格式层、因果层、终止层、奖励/评测层。格式层要求 action 能被解析并符合 schema；因果层要求 action 有对应 observation，并且后续判断真的基于 observation；终止层要求系统知道何时停止、何时提交、何时失败；奖励/评测层要求被打分的对象和被学习的对象是同一件事。



Insight. 轨迹完整性的核心不是“不要失败”。失败轨迹也很有价值。真正的问题是：这条失败轨迹是否清楚地告诉系统哪里失败、为什么失败、失败后状态是什么。

3 一个最小轨迹例子

为了把抽象概念落地，可以看一个最小 repo repair 轨迹。用户给出 issue，模型先读取相关文件，再运行失败测试，接着编辑代码，最后运行验证并提交 patch。这个过程看起来只是几轮消息，但每一步都带着不同的完整性条件。

Step	动作	必须记录	完整性问题
0	task intake	issue、repo、base commit、allowed tools	任务边界是否清楚
1	read file	path、command、stdout、return code	是否读到正确版本
2	run test	command、失败日志、环境状态	failure 是否可观测
3	edit patch	diff、tool args、文件状态变化	patch 是否真实应用
4	validate	测试命令、结果、耗时、异常	oracle 是否稳定
5	submit	final diff、exit status、evidence	reward boundary 是否明确

如果第 2 步的测试失败日志缺失，第 3 步的 edit 就缺少因果来源；如果第 3 步的 diff 没有记录，只记录了“我修改了文件”，后续训练无法知道模型到底改了什么；如果第 5 步没有 exit status，评测器

无法区分“提交了但测试失败”和“没有完成任务”。这些不是记录细节的洁癖，而是决定这条轨迹能否被复用。

同一条轨迹在不同系统里的使用方式不同。做 SFT 时，系统可能只把 action 作为预测目标；做 RL 时，系统需要用最终验证结果和中间 penalties 训练策略；做 failure diagnosis 时，系统需要检查哪一步第一次违反约束；做 product analytics 时，团队关心哪个工具最常失败、哪里最耗时。一个字段缺失，可能只影响其中一个用途，也可能让所有用途同时变差。

字段	SFT / RL 用途	诊断 / 产品用途
tool schema	确定 action target 是否合法	解释 schema 是否诱导错误
action args	学习参数生成和 API 使用	定位缺参、错参、路径错误
observation	学习基于反馈继续行动	检查工具失败和环境异常
state delta	判断修改是否真实发生	replay、审计、回滚
exit status	reward 和样本标签	统计成功、失败、超时、拒绝

Insight. 一个字段是否重要，不取决于它在日志里是否好看，而取决于它会不会被训练、评测、replay、debug 或产品统计重新解释。

4 格式错误怎样污染学习信号

工具调用格式错误看起来像局部语法问题，例如 JSON 不能解析、工具名不存在、缺少必填参数、同时输出 action 和 answer、没有按要求包在指定标签里。它实际影响的是动作空间。模型不是在抽象地“想使用工具”，而是在学习把自然语言意图映射到某个外部系统可执行的表示。表示一旦坏掉，环境不会执行，observation 不会产生，后续轨迹也会失去因果基础。

Qwen3-Coder-Next 的训练设计把这一点处理得很直接：多轮 agentic coding 轨迹由多个 scaffold 生成，之后用规则过滤 missing termination signals、task failures 和 malformed tool calls；在 CLI/IDE 场景里，还专门把 tool-call format correctness 当作训练目标，并在 RL 中对 invalid tool calls 关联 token 施加惩罚。这个设计说明 tool format 不是表面排版，而是 agentic coding 能否稳定执行的底线。

错误类型	表面现象	学习层后果
无工具调用	模型只解释、不行动	学到拖延或逃避执行
JSON 无效	parser 无法恢复参数	监督目标不可执行
未知工具	schema 与动作空间不一致	训练了不存在的接口
缺少参数	环境不能执行或执行错对象	observation 与意图断开
多 action 混杂	一轮输出不可判定	后续 observation 归因混乱

mini-SWE-agent 的实现给了一个很小但清楚的例子。它的 tool-call parser 在没有 tool call、参数 JSON 无法解析、工具名不是 bash 或缺少 command 时抛出 FormatError；regex action 模式也要求每轮恰好一个 action。这个错误不是被吞掉，而是以带有 interrupt_type 的消息进入轨迹。这样做的意义是保留协议边界：模型看到自己哪里没有满足 harness 要求，训练或调试系统也能把格式失败和任务失败区分开。

Insight. FormatError 应该成为 trajectory 的一部分，而不是悄悄消失。被吞掉的格式错误会让下一步看起来像无缘无故失败；被记录下来的格式错误才能成为可学习的反馈。

4.1 坏轨迹不一定都要删除

格式失败进入系统以后有三种处理方式：删除、修复、保留为负例。删除适合不可恢复的脏数据，例如缺少 action 原文、缺少 observation、无法知道最终状态；修复适合机械错误，例如 wrapper 缺

失但 JSON 内容完整，或者日志里能明确恢复 tool name 和参数；负例适合训练模型识别边界，例如明确标注为 FormatError、InvalidInvocation 或 RejectedAction 的样本。

处理方式	适合情况	风险
直接删除	无法恢复原始动作、状态或结果	可能丢掉真实失败分布
规则修复	格式局部坏、语义可恢复	修复器可能引入假结构
保留负例	错误类型清楚、反馈可学习	需要明确标签，不能混成正例
降权使用	样本有价值但不完全可信	权重策略难以解释

最危险的是第四种隐式处理：既不删除，也不标错，直接把坏轨迹当作普通成功或普通失败样本混入训练。这样模型看到的是含混监督：有时 malformed call 也出现在“可接受”的上下文里，有时没有 finish 也能得到最终 reward，有时缺少 observation 也继续推理。长期看，这会让模型把协议边界学软。

5 参数和顺序：比工具名更容易坏

在真实 tool use 中，选对工具只是第一步。很多任务真正困难的地方是参数、顺序和依赖。GTA 把工具调用过程拆成 instruction accuracy、tool accuracy、argument accuracy、summary accuracy 和最终 answer accuracy，并指出 argument prediction 往往是瓶颈。这个现象很重要：模型可能会选对工具，但把参数名、参数值、格式或引用对象写错，最终结果仍然失败。

TRAJECT-Bench 从另一个角度说明同一件事。它不只看 final answer，而是看 trajectory exact match、inclusion、tool usage 和 trajectory satisfaction。也就是说，评测需要知道工具是否被正确选择、是否以正确顺序调用、参数是否满足 schema、依赖关系是否成立。随着工具数量从短链走向中等长度，模型表现会明显下降；常见失败包括相似工具混淆、参数盲选、冗余调用和对间接意图理解不足。

层次	具体问题	为什么 final answer 不够
工具选择	选错相似 API 或遗漏必要工具	final answer 只显示失败，不说明错选在哪一步
参数生成	ID、路径、枚举值、时间范围错误	工具名正确但 execution 不成立
顺序依赖	先用 B 的结果才能调用 C	缺一个 observation 会导致后续全错
并行结构	可以同时查多个无依赖工具	过度串行会增加成本和错误传播
冗余调用	调了相关但无用的工具	提高 latency/cost，还可能污染上下文

这也解释了为什么 trajectory integrity 不能停留在“format parseable”。一个 parseable call 仍然可能是语义错误的 call。比如工具名合法、JSON 合法，但参数引用了不存在的 file path；或者第一步查出了 package name，第二步却把它当成 repository owner 使用。这类错误只有在 trajectory 级别才能被看见，因为它依赖前后步骤的关系。

6 Observation 和状态变化

Action 只有在接上 observation 后才成为可学习的经验。没有 observation，模型不知道外部世界如何回应；评测器不知道 action 是否真的执行；replay 系统不知道状态如何变化；debugger 也无法判断错误从哪里开始。对 coding agent 来说，observation 可能是 shell stdout/stderr、return code、文件 diff、测试结果、浏览器截图、API 响应、用户拒绝或工具错误。

OpenHands 的事件模型展示了更平台化的处理方式：conversation state 把 events 作为持久日志，view 通过 replay 新事件增量更新；系统还会寻找没有对应 observation 的 action。若 tool call 在 interrupt

时已经进入历史但没有结果，后续 provider 可能拒绝这个 conversation history，因此系统需要补 synthetic error observation 或 user rejection observation。这里的重点不是 OpenHands 的具体 API，而是一个普遍原则：action 和 observation 必须配对，否则轨迹的状态语义不完整。

缺口	轨迹看起来像什么	应该如何处理
action 无 observation	模型请求了工具，但历史里没有结果	补错误 observation 或标记 pending/rejected
observation 无 action 状态未记录 错误被吞掉	环境结果找不到触发动作 diff、cwd、文件版本缺失 下一轮只有含糊失败	标记为环境事件，避免归因给模型 记录 state delta 或 checkpoint 记录 tool error、return code 和异常类型
副作用不明	replay 可能改变真实系统	标注幂等性和恢复条件

mini-SWE-agent 的 trajectory 文件也体现了同一原则。它记录 full history、configuration、metadata、exit status 和 submission；每个 observation 可以携带 return code、raw output、timestamp 和 exception information。这样，一个失败运行不只是“没修好”，而是有足够结构让人判断是命令失败、格式失败、环境失败还是提交失败。

6.1 Replay 完整性和训练完整性不同

Replay 关心“能不能把过去的行为重新执行一遍”；训练完整性关心“模型从这段行为里应该学到什么”。二者相关，但不是同一件事。一个 action log 可能足够 replay，因为它记录了命令和参数；但它不一定适合训练，因为没有记录模型为什么选择这个 action、看到了什么上下文、失败后有没有收到可解释反馈。反过来，一段 SFT 轨迹可能足够训练格式和策略，但不适合 replay，因为缺少 base commit、环境变量、外部 API 状态或文件系统快照。

OpenHands 的 rerun 注释提醒了这一点：重新执行 event history 时，文件操作、terminal 命令、API 调用和浏览器状态可能不幂等，环境也可能已经改变。因此，不能把“有 event log”直接等同于“可安全 replay”。对 coding agent 的生产系统来说，trajectory 应该显式标出哪些 action 是只读的、哪些 action 有副作用、哪些 action 可以在 reset workspace 后重放、哪些 action 只能作为历史证据不能重放。

目标	必要字段	额外约束
训练格式	prompt、schema、action、observation、label	action target 必须清楚
评测判分 失败诊断	final output、tests、trajectory metrics step index、tool output、state、violation evidence	oracle 和版本稳定 需要因果链和证据窗口
安全 replay	base state、workspace snapshot、idempotency、permissions	需要隔离和回滚
产品审计	actor、time、permission、side effect、approval	需要责任边界

因此，设计 trajectory schema 时不要只为一个消费者服务。只为训练设计的轨迹，后续难以 debug；只为 replay 设计的轨迹，可能缺少监督标签；只为 UI 展示设计的轨迹，通常会丢掉机器可读细节。一个成熟系统会同时保留人能读的摘要和机器能验证的结构字段。

7 终止信号是 reward boundary

终止信号经常被低估。很多人把 finish action 当成 UI 细节：模型最后说“完成了”或者提交 patch。实际上，终止信号定义了评测和训练的边界。没有明确 stop，系统不知道应该继续让模型探索、判定超时、评估 patch，还是把当前状态视为未完成。对 RL 来说，终止还决定 reward 应该赋给哪条 rollout。

Qwen3-Coder-Next 的 RL 部分使用 unfinished trajectory penalty：当交互轮数超过预设上限或未能终止时，对 trajectory reward 施加惩罚。这不是为了让模型“少说话”，而是为了让模型学会在可验证状态下收束。没有这类信号，模型可能不断探索、反复调用工具、迟迟不提交，或者在还没有足够证据时过早给出 answer。

终止问题	运行层表现	训练/评测后果
没有 finish	rollout 达到 turn limit	reward 难以归因，成本不可控
过早 finish	未运行验证就提交	pass/fail 与推理过程断裂
假 finish	声称完成但无 patch 或证据	final answer 污染评测
多次 finish	输出多个候选终局	evaluator 不知道判哪一个
无提交元数据	不知道 patch、answer 或状态	轨迹不能复用为监督样本

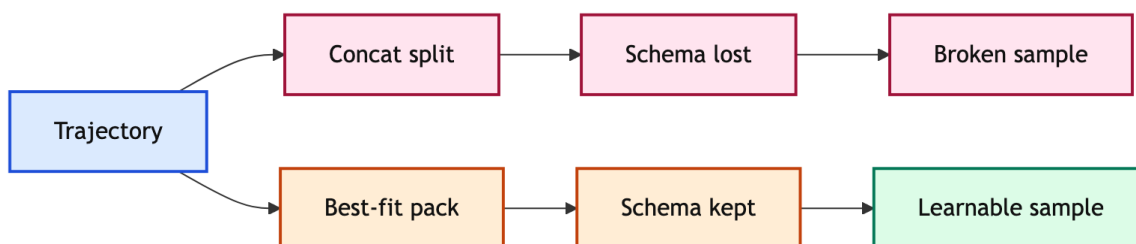
GTA 中的 no-action failure 也能从终止角度理解：模型既不调用工具，也不给 final answer，轨迹就卡在不产生可判定动作的状态。这个失败不是“回答差一点”，而是协议没有推进。对于 coding agent，类似情况包括一直解释可能原因、反复 ask user、不断查看文件但不提交 patch。终止信号让系统能区分“还在合理搜索”和“已经失去进展”。

Insight. 终止信号不是礼貌用语，而是 reward boundary。没有稳定终止，agent 的成功、失败、成本和学习信号都会变得模糊。

8 Best-Fit Packing 与上下文碎片

Trajectory integrity 也会在数据 packing 阶段被破坏。传统 concat-then-split 会把许多文档或轨迹拼成一条长序列，再按固定长度切块。对普通文本，这主要是上下文连续性问题；对 tool-call trajectory，它可能直接切断 schema 和 action 的关系。很多工具定义、调用格式、response wrapper 只在轨迹开头出现。如果后面的 action 被切到新样本里，而 tool definition 留在前一个样本，模型看到的就没有协议来源的调用。

Qwen3-Coder-Next 采用 best-fit packing，并在附录里明确指出：多轮 tool-call 任务通常在开头定义工具和调用格式，随机 chunking 会阻止模型学习严格的 tool invocation requirements。实验也显示，减少 fragmentation 会带来更稳定的长程任务收益。这里的机制不复杂：训练样本必须保留“规则 -> 行为 -> 反馈”的局部完整性。



这对后续批量制造 agent 数据有直接启发。不能只问 token utilization 是否高，还要问 packing 后的样本是否仍然有语义闭包。一个样本如果包含 action，却没有 schema；包含 observation，却没有 action；包含 final answer，却没有前面的证据，那么它虽然填满了 context window，却破坏了训练目标。

Packing 检查	合格样本	风险样本
Schema 可见	action 前能看到工具定义和格式	action 出现但 schema 被切掉
依赖可见	多轮引用能追溯前文 anchor	后一轮引用前一轮但前文缺失
结果可见	final answer 有 observation 支撑	只剩 answer 或只剩 observation
长度处理	超长轨迹按边界切分或丢弃	任意 token 边界切碎

9 多格式模板与 scaffold transfer

真实 coding agent 不共享同一种 tool-call 模板。有的系统用 JSON function call，有的用 XML-like tags，有的把工具定义放进 system prompt，有的把 observation 包成特定 role，有的 CLI/IDE scaffold 还允许用户自定义工具 schema。单一模板训练出来的模型，可能在熟悉格式里表现很好，但一换 scaffold 就开始输出 malformed calls。

Qwen3-Coder-Next 在这点上给了很明确的设计：不同 CLI/IDE scaffold 有不同 tool-calling schemas，因此训练数据覆盖多个 scaffold、任务类型、语言和用户交互模式；并且使用 21 个 tool chat templates，减少对单一结构的过拟合。换句话说，format robustness 是 deployment transfer 的一部分，不是 benchmark polish。

格式维度	常见变化	对模型的要求
工具定义	JSON schema、文本说明、混合模板	识别动作空间来源
调用包装	function call、XML、markdown block	生成可解析调用
返回包装	tool role、observation text、日志块	正确读取结果
多工具规则	串行、并行、每轮一个或多个	遵守执行协议
终止格式	finish action、answer tag、submission	清楚收束轨迹

UniToolCall 的 QAOA 表示也体现了同一个方向：把 Query、Action、Observation、Answer 做成统一结构，并在 function-call、turn、conversation 三个粒度评估。它不是说所有系统都应该长得一样，而是说训练和评测需要一个可对齐的中间表示，否则不同数据源之间的 schema、argument、observation 和评测脚本无法共同训练。

10 轨迹级评测

只看 final answer 会压缩掉太多信息。一个 agent 可能调用了错误工具但凭内部知识猜对答案；也可能工具路径完全正确，但最后 summary 写错；还可能最终 patch 通过测试，但中间使用了泄漏信息或无关 API。Trajectory-level metrics 的价值是把最终结果拆回过程：工具是否选对、参数是否正确、顺序是否满足依赖、结果是否基于 observation、调用数量是否合理。

评测方式	衡量对象	适合回答的问题
Final accuracy	最终答案或测试是否通过	任务是否成功
Exact match	工具序列是否完全一致	是否复现 gold trace
Inclusion	必要工具是否被包含	是否遗漏关键步骤
Tool usage	参数、格式、值约束是否满足	是否可执行且语义正确
Trajectory satisfy	无 gold trace 时整体是否合理	是否存在可接受替代路径
Critical step	第一个不可恢复错误在哪	失败如何修复或过滤

Strict 和 flexible metrics 各有作用。Strict metric 要求每个 function call 都正确，适合验证协议底线和 benchmark 可比性；flexible metric 给部分正确的调用计分，适合分析模型在长轨迹中的渐进能力。对训练来说，两者都需要：strict gate 防止坏格式进入数据，flexible signal 帮助定位哪些子能力需要改进。

10.1 不要把 trajectory metric 当作单一真相

Trajectory metric 的目的不是取代 final accuracy，而是解释 final accuracy。Exact match 太严格时，会惩罚可行但不同的路径；inclusion 太宽时，会容忍冗余调用和错误顺序；tool usage 依赖参数 gold label，如果 gold trace 本身不是唯一正确路径，就需要 trajectory satisfaction 或人工/LLM judge 辅助。好的评测报告应该同时给出最终结果和过程指标，而不是把它们压成一个分数。

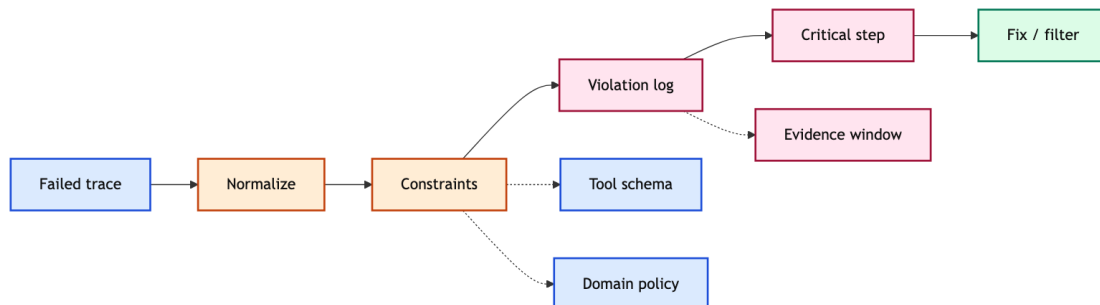
对 coding agent 来说，这一点尤其重要。一个 patch 可以通过测试，但中间读错日志、调用无关工具、浪费大量预算；另一个 patch 没通过测试，但定位、编辑和验证路径都合理，只差一个边界条件。前者适合做产品成本优化或 reward-hacking 检查，后者适合做训练数据或 curriculum。单一 final score 会把这两种样本混在一起。

分数形态	它能说明什么	它不能说明什么
最终通过率	任务是否解决	解决路径是否可靠
过程精确匹配	是否复现 gold trace	是否存在替代正确路径
参数正确率	API 调用是否可执行	是否选了最经济策略
轨迹满意度	整体路径是否合理	judge 是否稳定、是否可复现
失败定位准确率	root cause 是否找到	修复策略是否一定有效

Insight. Final answer 是结果压缩，trajectory metric 是过程展开。没有过程展开，系统很难知道应该改模型、改工具 schema、改 retrieval、改 reward，还是改 evaluator。

11 失败定位：从 terminal failure 到 critical step

复杂 agent 经常不是在最后一步突然失败，而是在早期某一步产生不可恢复偏差，后面只是把这个偏差放大。AgentRx 把 critical failure 定义为 first unrecoverable failure，并把失败轨迹规范化为包含 messages、tool calls、tool outputs 和 observable environment state 的结构。它再从 tool schema、domain policy 和 trajectory prefix 合成 constraints，逐步产生 violation log，让 judge 定位关键失败步骤。



这个思想对 coding agent 很有用。一个 patch 最后测试失败，原因可能是定位错文件、误读 stack trace、选择了错误编辑点、运行了不相关测试、忽略了 tool error，或者提交时遗漏了 diff。只看 terminal failure，会把这些全压成“patch failed”。看 trajectory，才能区分第一个不可恢复错误和后续 cascading failures。

失败类别	轨迹证据	修复方向
Plan adherence failure	跳过必要步骤或执行额外动作	加强约束和 stop gate
Invalid invocation	参数缺失、schema validation 失败	改 tool schema 或格式训练
Ungrounded claim	回答不基于 observation	加 observation grounding check
Misread output	tool result 被错误解释	加 result parser 或 evidence card
System failure	工具/环境异常而非模型错误	隔离环境并标注非模型失败

失败定位还有一个产品层意义：它让 bad trajectory 变成可处理对象。如果关键错误是 malformed call，就进入 format training 或过滤；如果是 environment failure，就不该惩罚模型；如果是 redundant call，就优化 tool retrieval 或 cost policy；如果是 early finish，就调整 termination reward。没有 step-indexed evidence，这些改进只能靠猜。

12 工程检查表

把 trajectory integrity 做进系统，不是增加一个总开关，而是在数据、运行、评测三个阶段设置不同 gate。训练数据要保证样本有 schema、action、observation 和 stop；运行时要保证 parser、executor 和 logger 不吞错误；评测时要保证 final score 能回溯到轨迹过程。

阶段	必须检查	不合格时
数据生成	tool schema、调用格式、observation、finish	filter、repair 或降级为失败样本
数据 packing	schema 与后续 calls 是否同样本可见	BFP、按轨迹边界切分或丢弃
SFT 训练	action target 是否可解析、参数是否有效	不作为正例学习
RL rollout	turn limit、finish、tool-format penalty	施加未完成或格式惩罚
Runtime	orphan action、tool error、rejection、return code	写入 observation 或 error event
Evaluation	final 与 trajectory metrics 同时报告	拆分失败归因

Gate 1：格式先于语义。

不可解析的 action 不应该进入语义评判。先判断它是否满足 tool-call protocol，再判断它是否是正确工具、正确参数、正确顺序。这样可以避免把 parser failure 错判为 reasoning failure。

Gate 2：action 必须有后果。

每个外部 action 都应该能找到 observation、rejection 或 error。若 action 处于 pending 状态，轨迹需要显式记录，而不是让下一轮模型面对孤立 tool call。

Gate 3：终止必须可判定。

每条轨迹都要有明确 exit status：submitted、answered、failed、limits exceeded、user rejected、environment failed。模糊终止会污染 reward 和统计。

Gate 4：数据切分不能破坏协议。

packing、truncation 和 compaction 都不能把 schema、history anchor、observation 和 answer 随机拆散。效率优化不能以破坏可学习性为代价。

12.1 上线前验收

如果把 trajectory integrity 做成生产 gate，可以用下面的验收列表。它不是为了追求形式完美，而是为了避免系统在训练、评测和产品运行里使用含义不明的数据。

验收项	通过标准	不通过信号
Parser coverage	所有 action 都能归类为 valid/error/rejected	存在未知格式沉默通过
Action-observation pairing	每个 action 有 observation、error 或 pending 标记	orphan tool call 留在历史里
Exit status taxonomy	success/failure/timeout/reject/env-failure 可区分	只有一个模糊 done 字段
Version grounding	base commit、tool version、schema version 可追溯	replay 或评测读不到版本
Reward explainability	reward 能映射到 final 和中间 penalties	只有总分没有原因
Metric dashboard	final/process/cost/error 分开报告	只有 pass rate

这个列表还有一个隐藏价值：它帮助团队决定问题归属。Parser coverage 失败通常是模型格式或 scaffold 协议问题；pairing 失败通常是 runtime 或 event log 问题；exit taxonomy 失败通常是 product workflow 问题；version grounding 失败通常是 environment reproducibility 问题；reward explainability 失败通常是 RL 或 benchmark 设计问题。把这些混在一起，会让“agent 不行”变成一个无法行动的结论。

13 局限和权衡

严格的 trajectory integrity 会带来成本。第一，过滤过严可能丢掉一些真实世界里“格式不完美但可恢复”的行为，导致模型对 messy environment 适应不足。第二，统一表示会掩盖不同 scaffold 的细节；如果所有数据都被规整成一种 QAOA，模型可能不再熟悉真实产品里的各种 wrapper 和 edge case。第三，trajectory metric 本身也可能过拟合：模型学会满足 exact match，却不一定学会更好的替代路径。

因此，合理做法不是把所有轨迹都强制成同一个模板，而是区分三个层次：底层 protocol 必须清楚，格式多样性必须保留，评测指标必须能解释。Qwen 的多模板训练、UniToolCall 的统一表示、TRAJECT-Bench 的 trajectory-aware metrics 和 AgentRx 的失败定位，其实分别对应这三件事：让格式足够稳、让表示足够可比、让过程足够可诊断。

最终 takeaway:

高质量 coding agent 不是只需要更多成功轨迹，也需要更干净的失败轨迹。
只要 action、observation、state、stop 和 evidence 的关系保持完整，失败也能变成训练信号、诊断信号和系统改进信号。

13.1 读者复盘清单

读完这一讲，可以用六个问题快速审视任何 agent 系统的 trajectory 设计：

问题	合格回答应该包含什么
动作空间是否明确	每个 action 都能追溯到 tool schema、格式规则和权限边界。
反馈是否闭合	每个 action 都有 observation、error、rejection 或 pending 标记。
状态是否可解释	文件、环境、浏览器、API 或 workspace 的变化能被定位到具体 step。
终止是否可判定	success、failure、timeout、reject、environment failure 有不同 exit status。
训练是否不被污染	malformed call、缺失 observation、坏 finish 不会混成普通正例。
评测是否可诊断	final score 之外还有 tool、argument、order、cost、critical-step 等过程信号。

参考资料

- [1] [Qwen3-Coder-Next Technical Report](#). multi-scaffold trajectory generation, tool-call filtering, BFP, template diversity, RL penalties.
- [2] [GTA: A Benchmark for General Tool Agents](#). fine-grained instruction/tool/argument/summary/final-answer metrics and error analysis.
- [3] [UniToolCall](#). unified QAOA representation, executable-supervision filtering, anchor linkage, strict/flexible metrics.
- [4] [TRAJECT-Bench](#). trajectory-aware evaluation for tool selection, parameters, order, dependency, and final performance.
- [5] [AgentRx](#). failed-trajectory normalization, critical failure step, auditable violation logs, failure taxonomy.
- [6] [mini-SWE-agent](#). linear trajectory files, FormatError handling, observation and exit-status metadata.
- [7] [OpenHands SDK](#). event log, view replay, unmatched action handling, synthetic error observations, replay caveats.